

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Домашняя работа №2
«Проектирование и технический дизайн API»

Выполнил:

Фамилия Имя

Группа

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Спроектировать API бэкенд-приложения для выбранного варианта лабораторного проекта. Для этого необходимо:

- описать все функциональные и нефункциональные требования к бэкенд-приложению;
- выбрать формат реализации API: REST API или Backend For Frontend;
- спроектировать все эндпоинты API в формате OpenAPI-схемы с учётом ранее разработанной диаграммы БД;
- описать тело каждого запроса и тело каждого ответа, продумать возможные ошибки, возвращаемые из API.

Тема проекта: вариант 1 – приложение для бронирования столиков в ресторанах.

Ход работы

1. Требования к приложению

1.1 Функциональные требования

Функциональные требования приведены в таблице 1. Гостем называется пользователь без токена авторизации.

Таблица 1 – Функциональные требования

№	Требование	Роль
FR1	Регистрация нового пользователя	Гость
FR2	Вход по email и паролю, получение токена	Гость
FR3	Личный кабинет: просмотр и изменение профиля	USER
FR4	Поиск ресторанов с фильтрацией по кухне, расположению (город, район) и цене	Гость
FR5	Страница ресторана: описание, фотографии, меню, отзывы	Гость
FR6	Просмотр свободных столиков на выбранные дату и временной слот	Гость
FR7	Бронирование столика на дату и слот с указанием числа гостей	USER
FR8	История бронирований пользователя, отмена своей брони	USER
FR9	Отзывы о ресторанах: создание, изменение, удаление	USER

№	Требование	Роль
FR10	Управление рестораном: описание, фотографии, меню, столики, слоты	RESTAURANT_ADMIN, ADMIN
FR11	Просмотр броней ресторана, их подтверждение и отмена	RESTAURANT_ADMIN, ADMIN
FR12	Управление ресторанами и справочниками городов и кухонь, назначение администраторов ресторанов	ADMIN

1.2 Нефункциональные требования

Нефункциональные требования приведены в таблице 2.

Таблица 2 – Нефункциональные требования

Категория	Требование
Безопасность	Аутентификация по JWT-токену; пароли хранятся только в виде хэша; права проверяются по роли пользователя, а администратор ресторана управляет только своими ресторанами.
Производительность	Списки возвращаются постранично (параметры page и limit, не более 100 записей на странице); фильтрация и сортировка выполняются на сервере; рейтинг ресторана хранится в таблице Restaurant для быстрой сортировки.
Целостность данных	Ограничения из схемы БД (уникальность, допустимые диапазоны значений) отражаются в API; конфликтные операции возвращают ошибку 409; двойное бронирование одного столика на слот и дату невозможно.
Единообразие ошибок	Все ошибки возвращаются в едином формате {statusCode, message, errors?} с кодами 400, 401, 403, 404, 409.
Форматы данных	Обмен данными в формате JSON (UTF-8); имена полей в snake_case, как в БД; даты в формате ISO 8601 (ГГГГ-ММ-ДД), время в формате ЧЧ:ММ.
Версионирование	Версия API указывается в адресе: /api/v1.
Документируемость	API полностью описан в спецификации OpenAPI 3.0.3, которую можно просматривать в Swagger.

1.3 Роли и права доступа

В системе выделены три роли, которые хранятся в поле User.role, и гость без авторизации. Права ролей приведены в таблице 3.

Таблица 3 – Роли и права

Роль	Кто это	Возможности
Гость	Пользователь без токена	Регистрация и вход; поиск и просмотр ресторанов, меню, фотографий, отзывов, слотов и свободных столиков.
USER	Зарегистрированный пользователь	Всё, что доступно гостю, а также личный кабинет, бронирование, история и отмена своих броней, отзывы.
RESTAURANT_ADMIN	Администратор одного или нескольких ресторанов	Управление данными своих ресторанов (описание, фото, меню, столики, слоты), просмотр, подтверждение и отмена броней этих ресторанов.
ADMIN	Администратор системы	Все действия, включая создание и удаление ресторанов, управление справочниками и назначение администраторов ресторанов.

2. Выбор формата API

Для реализации API рассмотрены два подхода: REST API и Backend For Frontend (BFF). Сравнение приведено в таблице 4.

Таблица 4 – Сравнение REST и BFF

Критерий	REST API	Backend For Frontend
Суть подхода	Ресурсы (рестораны, брони, отзывы) и стандартные методы HTTP	Отдельный слой API под конкретный клиент, который собирает данные из нескольких источников
Число клиентов	Подходит для одного клиента или нескольких с похожими потребностями	Оправдан, если клиенты (веб, мобильное приложение) требуют разных данных
Архитектура	Монолитное приложение или небольшое число сервисов	Обычно при микросервисах, чтобы клиенту не обращаться к каждому сервису отдельно
Сложность	Ниже: один слой, понятная структура адресов	Выше: дополнительный слой, который нужно разрабатывать и поддерживать
Документирование	Одна спецификация OpenAPI	Отдельная спецификация для каждого BFF

Выбран REST API. В проекте один клиент, поэтому отдельный слой под клиента не нужен. Приложение разрабатывается как монолит, данные хорошо описываются ресурсами из диаграммы БД: рестораны, столики,

слоты, бронирования, отзывы. REST проще проектировать и реализовывать на Express в ЛР1, а его спецификация напрямую используется для тестирования в Postman в ДЗЗ.

3. Общие принципы проектирования

- **Базовый адрес и версия.** Все запросы отправляются на `http://localhost:3000/api/v1`, версия API указана в адресе.
- **Ресурсы.** Вложенные ресурсы принадлежат родителю (`/restaurants/{restaurantId}/menu`). Действие определяется HTTP-методом, а не адресом.
- **Методы и коды успеха.** GET – получение (200), POST – создание (201), PATCH – частичное изменение (200), DELETE – удаление (204). Метод PUT не используется: все изменения частичные.
- **Формат данных.** JSON, имена полей в `snake_case` как в БД, идентификаторы – целые числа, даты в формате ГГГГ-ММ-ДД, время в формате ЧЧ:ММ.
- **Авторизация.** После входа клиент получает JWT-токен и передаёт его в заголовке `Authorization: Bearer <токен>`. Без токена защищённые методы возвращают 401, при нехватке прав – 403.
- **Пагинация.** Списки, которые могут быть большими (поиск ресторанов, отзывы, история броней, брони ресторана), принимают параметры `page` (по умолчанию 1) и `limit` (по умолчанию 20, максимум 100) и возвращают объект `{items, meta}`, где `meta` содержит `total`, `page`, `limit` и `total_pages`.
- **Фильтрация и сортировка.** Задаются параметрами запроса (`query`), например `cuisine_id`, `city_id`, `min_price`, `sort`.

4. Структура спецификации OpenAPI

Спецификация оформлена в файле `openapi.yaml` (OpenAPI 3.0.3). Блоки файла описаны в таблице 5.

Таблица 5 – Блоки спецификации

Блок	Содержимое
openapi, info	Версия спецификации (3.0.3), название и версия API, общее описание: авторизация, роли, пагинация, формат ошибок.
servers	Адрес сервера: <code>http://localhost:3000/api/v1</code> .

Блок	Содержимое
tags	12 групп эндпоинтов, по которым Swagger раскладывает документацию.
paths	26 адресов и 43 операций. У каждой операции: tags, summary, description, operationId, parameters, requestBody, security, responses.
components.securitySchemes	Схема авторизации bearerAuth (JWT в заголовке Authorization).
components.parameters	12 повторяющихся параметров: идентификаторы в адресе, page и limit.
components.responses	5 типовых ответов с ошибками: BadRequest, Unauthorized, Forbidden, NotFound, Conflict.
components.schemas	44 схем данных: сущности, тела запросов, страницы списков, перечисления.

Повторяющиеся элементы вынесены в блок components и подключаются ссылками (\$ref), поэтому каждая схема описана один раз. Для большинства сущностей введено несколько вариантов схемы: например, Restaurant описывает ответ сервера, RestaurantCreate – тело запроса на создание (без id и служебных дат), RestaurantUpdate – тело запроса на изменение (все поля необязательны, но нужно передать хотя бы одно), RestaurantPage – страницу списка в виде {items, meta}.

5. Описание эндпоинтов

Спроектировано 43 операций в 12 группах. Для каждой операции в спецификации описаны параметры, тело запроса, успешный ответ и возможные ошибки. Ниже приведены сводные таблицы по группам; в столбце «Доступ» указано, кому разрешена операция.

5.1 Авторизация и пользователи

Таблица 6 – Эндпоинты: авторизация и пользователи

Метод	Адрес	Назначение	Доступ
POST	/auth/register	Регистрация	Без авторизации
POST	/auth/login	Вход	Без авторизации
GET	/users/me	Личный кабинет: свой профиль	Авторизованный пользователь
PATCH	/users/me	Изменить свой профиль	Авторизованный пользователь
GET	/users/me/bookings	История бронирований пользователя	Авторизованный пользователь

5.2 Справочники городов и кухонь

Таблица 7 – Эндпоинты: справочники городов и кухонь

Метод	Адрес	Назначение	Доступ
GET	/cities	Список городов	Без авторизации
POST	/cities	Добавить город	ADMIN
DELETE	/cities/{cityId}	Удалить город	ADMIN
GET	/cuisines	Список кухонь	Без авторизации
POST	/cuisines	Добавить кухню	ADMIN
DELETE	/cuisines/{cuisineId}	Удалить кухню	ADMIN

5.3 Рестораны

Таблица 8 – Эндпоинты: рестораны

Метод	Адрес	Назначение	Доступ
GET	/restaurants	Поиск ресторанов с фильтрацией	Без авторизации
POST	/restaurants	Создать ресторан	ADMIN
GET	/restaurants/{restaurantId}	Страница ресторана	Без авторизации
PATCH	/restaurants/{restaurantId}	Изменить ресторан	ADMIN или администратор этого ресторана
DELETE	/restaurants/{restaurantId}	Удалить ресторан	ADMIN

Параметры поиска ресторанов (GET /restaurants) приведены в таблице 9. Они реализуют фильтрацию по кухне, расположению и цене из условия задания; параметры можно комбинировать.

Таблица 9 – Параметры запроса GET /restaurants

Параметр	Тип	Описание
q	строка	Поиск по названию ресторана
city_id	целое число	Фильтр по городу
district	строка	Фильтр по району
cuisine_id	целое число	Фильтр по кухне
min_price	число	Минимальный средний чек, руб.
max_price	число	Максимальный средний чек, руб.
min_rating	число	Минимальный рейтинг

Параметр	Тип	Описание
sort	одно из: rating_desc, price_asc, price_desc, name_asc (по умолчанию rating_desc)	Сортировка
page	целое число (по умолчанию 1)	Номер страницы
limit	целое число (по умолчанию 20)	Записей на странице (не более 100)

5.4 Фотографии и меню

Таблица 10 – Эндпоинты: фотографии и меню

Метод	Адрес	Назначение	Доступ
GET	/restaurants/{restaurantId}/photos	Фотографии ресторана	Без авторизации
POST	/restaurants/{restaurantId}/photos	Добавить фотографию	ADMIN или администратор этого ресторана
DELETE	/restaurants/{restaurantId}/photos/{photoId}	Удалить фотографию	ADMIN или администратор этого ресторана
GET	/restaurants/{restaurantId}/menu	Меню ресторана	Без авторизации
POST	/restaurants/{restaurantId}/menu	Добавить позицию меню	ADMIN или администратор этого ресторана
PATCH	/restaurants/{restaurantId}/menu/{menuItemId}	Изменить позицию меню	ADMIN или администратор этого ресторана
DELETE	/restaurants/{restaurantId}/menu/{menuItemId}	Удалить позицию меню	ADMIN или администратор этого ресторана

5.5 Отзывы

Таблица 11 – Эндпоинты: отзывы

Метод	Адрес	Назначение	Доступ
GET	/restaurants/{restaurantId}/reviews	Отзывы о ресторане	Без авторизации
POST	/restaurants/{restaurantId}/reviews	Оставить отзыв	Любой авторизованный пользователь
PATCH	/reviews/{reviewId}	Изменить свой отзыв	Автор отзыва
DELETE	/reviews/{reviewId}	Удалить отзыв	Автор отзыва или ADMIN

5.6 Столики, слоты и свободные столики

Таблица 12 – Эндпоинты: столики, слоты и свободные столики

Метод	Адрес	Назначение	Доступ
GET	/restaurants/{restaurantId}/tables	Столики ресторана	Без авторизации
POST	/restaurants/{restaurantId}/tables	Добавить столик	ADMIN или администратор этого ресторана
PATCH	/restaurants/{restaurantId}/tables/{tableId}	Изменить столик	ADMIN или администратор этого ресторана
DELETE	/restaurants/{restaurantId}/tables/{tableId}	Удалить столик	ADMIN или администратор этого ресторана
GET	/restaurants/{restaurantId}/time-slots	Временные слоты ресторана	Без авторизации
POST	/restaurants/{restaurantId}/time-slots	Добавить слот	ADMIN или администратор этого ресторана
PATCH	/restaurants/{restaurantId}/time-slots/{timeSlotId}	Изменить слот	ADMIN или администратор этого ресторана
DELETE	/restaurants/{restaurantId}/time-slots/{timeSlotId}	Удалить слот	ADMIN или администратор этого ресторана
GET	/restaurants/{restaurantId}/availability	Свободные столики на дату и слот	Без авторизации

5.7 Администраторы ресторана

Таблица 13 – Эндпоинты: администраторы ресторана

Метод	Адрес	Назначение	Доступ
GET	/restaurants/{restaurantId}/admins	Администраторы ресторана	ADMIN или администратор этого ресторана
POST	/restaurants/{restaurantId}/admins	Назначить администратора ресторана	ADMIN
DELETE	/restaurants/{restaurantId}/admins/{userId}	Снять администратора ресторана	ADMIN

5.8 Бронирования

Таблица 14 – Эндпоинты: бронирования

Метод	Адрес	Назначение	Доступ
POST	/bookings	Забронировать столик	Любой авторизованный пользователь
GET	/bookings	Брони ресторана	ADMIN (все брони) или администратор ресторана (только брони своих ресторанов)
GET	/bookings/{bookingId}	Информация о брони	Владелец брони, администратор ресторана или ADMIN
PATCH	/bookings/{bookingId}/status	Изменить статус брони	Владелец (только отмена), администратор ресторана, ADMIN

6. Тела запросов и ответов

Тела запросов и ответов описаны схемами в блоке `components.schemas`. Схемы каждой сущности приведены в таблице 15.

Таблица 15 – Схемы запросов и ответов

Сущность	Ответ	Создание	Изменение	Список
Пользователь и вход	User, AuthResponse	RegisterRequest, LoginRequest	UserUpdate	–
Город	City	CityCreate	–	–
Кухня	Cuisine	CuisineCreate	–	–
Ресторан	Restaurant, RestaurantDetails	RestaurantCreate	RestaurantUpdate	RestaurantPage
Фотография	RestaurantPhoto	RestaurantPhotoCreate	–	–
Позиция меню	MenuItem	MenuItemCreate	MenuItemUpdate	–
Отзыв	Review	ReviewCreate	ReviewUpdate	ReviewPage
Столик	RestaurantTable	RestaurantTableCreate	RestaurantTableUpdate	–
Слот	TimeSlot	TimeSlotCreate	TimeSlotUpdate	–
Администратор ресторана	RestaurantAdmin	RestaurantAdminCreate	–	–
Бронирование	Booking, BookingDetails	BookingCreate	BookingStatusUpdate	BookingPage

Схемы согласованы с таблицами из ДЗ1. Идентификаторы и служебные даты (created_at, updated_at) назначает сервер, поэтому в схемах создания их нет, а в схемах ответа они помечены как только для чтения. Хэш пароля (password_hash) в API не передаётся: при регистрации и входе клиент отправляет поле password, а в ответах пользователь возвращается без пароля. Рейтинг ресторана (rating) вычисляется сервером и изменяется только через отзывы.

6.1 Примеры запросов и ответов

Регистрация. Запрос POST /auth/register:

```
{
  "email": "anna@example.com",
  "password": "Str0ngPass!",
  "first_name": "Анна",
  "last_name": "Иванова",
  "phone": "+79001234567"
}
```

Ответ 201 Created:

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "Bearer",
  "expires_in": 3600,
  "user": {
    "id": 5,
    "role": "USER",
    "email": "anna@example.com",
    "first_name": "Анна",
    "last_name": "Иванова",
    "phone": "+79001234567",
    "created_at": "2026-09-19T10:00:00Z",
    "updated_at": "2026-09-19T10:00:00Z"
  }
}
```

Поиск ресторанов. Запрос GET

/restaurants?city_id=1&cuisine_id=2&max_price=2500&sort=rating_desc&page=1&limit=10, ответ 200 OK:

```
{
  "items": [
    {
      "id": 5,
      "name": "Траттория Порто",
      "description": "Уютный ресторан итальянской кухни",
      "city_id": 1,
      "cuisine_id": 2,
      "district": "Центральный",
      "address": "Невский проспект, 10",
      "latitude": 59.9343,
      "longitude": 30.3351,
      "phone": "+78121234567",
      "average_check": 1800,
      "opening_time": "10:00",
      "closing_time": "23:00",
      "rating": 4.6,
      "is_active": true,
      "created_at": "2026-09-01T09:00:00Z",
      "updated_at": "2026-09-10T12:30:00Z"
    }
  ],
  "meta": { "total": 1, "page": 1, "limit": 10, "total_pages": 1 }
}
```

Свободные столики. Запрос GET

/restaurants/5/availability?date=2026-10-05&time_slot_id=2&guests_count=3, ответ 200 OK:

```
{
  "date": "2026-10-05",
  "time_slot": {
    "id": 2, "restaurant_id": 5,
    "start_time": "14:00", "end_time": "16:00", "is_active": true
  },
  "tables": [
    {
      "id": 3, "restaurant_id": 5, "number": "A1",
      "capacity": 4, "zone": "Терраса", "is_active": true
    }
  ]
}
```

Создание брони. Запрос POST /bookings (требуется токен):

```
{
  "restaurant_table_id": 3,
  "time_slot_id": 2,
  "booking_date": "2026-10-05",
  "guests_count": 3,
  "comment": "Столик у окна"
}
```

Ответ 201 Created:

```
{
  "id": 12,
  "user_id": 5,
  "restaurant_table_id": 3,
  "time_slot_id": 2,
  "status": "PENDING",
  "booking_date": "2026-10-05",
  "guests_count": 3,
  "comment": "Столик у окна",
  "created_at": "2026-09-19T10:05:00Z",
  "updated_at": "2026-09-19T10:05:00Z"
}
```

7. Обработка ошибок

Все ошибки возвращаются в едином формате: statusCode совпадает с HTTP-кодом, message кратко описывает причину, errors (только для ошибок валидации) содержит список ошибок по полям. Формат описан схемой Error, а типовые ответы вынесены в components.responses и подключаются ко всем операциям ссылкой. Пример ошибки валидации при регистрации:

```
{
  "statusCode": 400,
  "message": "Ошибка валидации",
  "errors": [
    { "field": "email", "message": "Некорректный формат email" },
    { "field": "password", "message": "Пароль должен содержать не менее 8 символов" }
  ]
}
```

Используемые коды ошибок приведены в таблице 16.

Таблица 16 – Коды ошибок

Код	Когда возвращается	Сообщение (пример)
400	Ошибка валидации: неверный формат или недопустимые значения (некорректный email, guests_count меньше 1, дата брони в прошлом, вместимость столика меньше числа гостей)	Ошибка валидации
401	Токен не передан или недействителен	Требуется авторизация
403	Прав недостаточно: пользователь USER создаёт ресторан, администратор изменяет чужой ресторан, пользователь отменяет чужую бронь	Доступ запрещён
404	Ресторан, столик, слот, бронь или другой объект не найден	Объект не найден
409	Конфликт с текущим состоянием данных (см. список ниже)	Конфликт данных

Конфликты (409). Эти ошибки связаны с ограничениями схемы БД из ДЗ1:

- email при регистрации уже занят (POST /auth/register);
- пользователь уже оставил отзыв на этот ресторан (POST /restaurants/{restaurantId}/reviews);
- столик на выбранные дату и слот уже забронирован (POST /bookings);
- номер столика или начало слота уже существуют в ресторане, слоты пересекаются;
- пользователь уже назначен администратором ресторана;
- удаляемый город, кухня, ресторан, столик или слот связан с другими записями: рестораны, брони, отзывы (соответствует ON DELETE RESTRICT в ДЗ1);
- недопустимый переход статуса брони (PATCH /bookings/{bookingId}/status).

Допустимые переходы статусов брони приведены в таблице 17.

Таблица 17 – Переходы статусов бронирования

Из статуса	В статус	Кто может выполнить
PENDING	CONFIRMED, CANCELLED	CONFIRMED – администратор ресторана или ADMIN; CANCELLED – владелец, администратор ресторана или ADMIN
CONFIRMED	COMPLETED, CANCELLED	COMPLETED – администратор ресторана или ADMIN; CANCELLED – владелец, администратор ресторана или ADMIN
CANCELLED	–	Конечный статус, изменение невозможно
COMPLETED	–	Конечный статус, изменение невозможно

8. Связь с диаграммой базы данных

Каждая таблица из ДЗ1 представлена в API схемами и эндпоинтами; соответствие приведено в таблице 18.

Таблица 18 – Соответствие таблиц БД и API

Таблица БД	Схемы API	Эндпоинты
User, Roles	User, UserShort, Role, RegisterRequest, LoginRequest, AuthResponse, UserUpdate	/auth/register, /auth/login, /users/me
City, Cuisine	City, Cuisine, CityCreate, CuisineCreate	/cities, /cuisines

Таблица БД	Схемы API	Эндпоинты
Restaurant	Restaurant, RestaurantShort, RestaurantDetails, RestaurantCreate, RestaurantUpdate, RestaurantPage	/restaurants
RestaurantAdmin	RestaurantAdmin, RestaurantAdminCreate	/restaurants/{restaurantId}/admins
RestaurantPhoto	RestaurantPhoto, RestaurantPhotoCreate	/restaurants/{restaurantId}/photos
MenuItem, MenuCategories	MenuItem, MenuCategory, MenuItemCreate, MenuItemUpdate	/restaurants/{restaurantId}/menu
Review	Review, ReviewCreate, ReviewUpdate, ReviewPage	/restaurants/{restaurantId}/reviews, /reviews/{reviewId}
RestaurantTable	RestaurantTable, RestaurantTableCreate, RestaurantTableUpdate	/restaurants/{restaurantId}/tables
TimeSlot	TimeSlot, TimeSlotCreate, TimeSlotUpdate	/restaurants/{restaurantId}/time-slots
Booking, BookingStatuses	Booking, BookingDetails, BookingCreate, BookingStatusUpdate, BookingStatus, BookingPage, Availability	/bookings, /users/me/bookings, /restaurants/{restaurantId}/availability

В API добавлено то, чего нет в таблицах БД в явном виде:

- вложенные объекты для удобства клиента: город, кухня и фотографии в странице ресторана (RestaurantDetails), краткие данные автора в отзыве, ресторан, столик и слот в брони (BookingDetails);
- пагинация списков (объект meta);
- поиск свободных столиков (Availability): результат вычисляется по таблицам RestaurantTable и Booking и в БД отдельно не хранится.

Не вошли в API: хэш пароля (password_hash) и служебная логика пересчёта рейтинга, которая выполняется на сервере при создании, изменении и удалении отзывов.

9. Демонстрация в Swagger

Спецификация оформлена в файле openapi.yaml, который прилагается к отчёту. Для просмотра файл открывается в Swagger Editor (editor.swagger.io)

Вывод

В ходе работы сформулированы функциональные и нефункциональные требования к приложению для бронирования столиков, в качестве формата API выбран REST и спроектировано API в виде спецификации OpenAPI 3.0.3: 43 операций в 12 группах, 44 схем данных, единый формат ошибок и авторизация по JWT-токену. Для каждого эндпоинта описаны параметры, тела запросов и ответов, коды успеха и возможные ошибки, а схемы согласованы с диаграммой базы данных из ДЗ1. Спецификация будет использована в следующих работах: по ней составляется тестовый сценарий в Postman (ДЗ3), а эндпоинты реализуются на Express и TypeORM (ЛР1).